

INFORME AVANZADO DE CALIDAD

Calculadora de Descuentos con Pytest y CI

Tecnologías: Pytest 9.0.3, Faker, Coverage y GitHub Actions

Fecha: 20 de mayo de 2026

Elaborado por: Equipo de Desarrollo (Desarrollador & Antigravity)

1. ¿Que es Pytest y por que lo usamos?

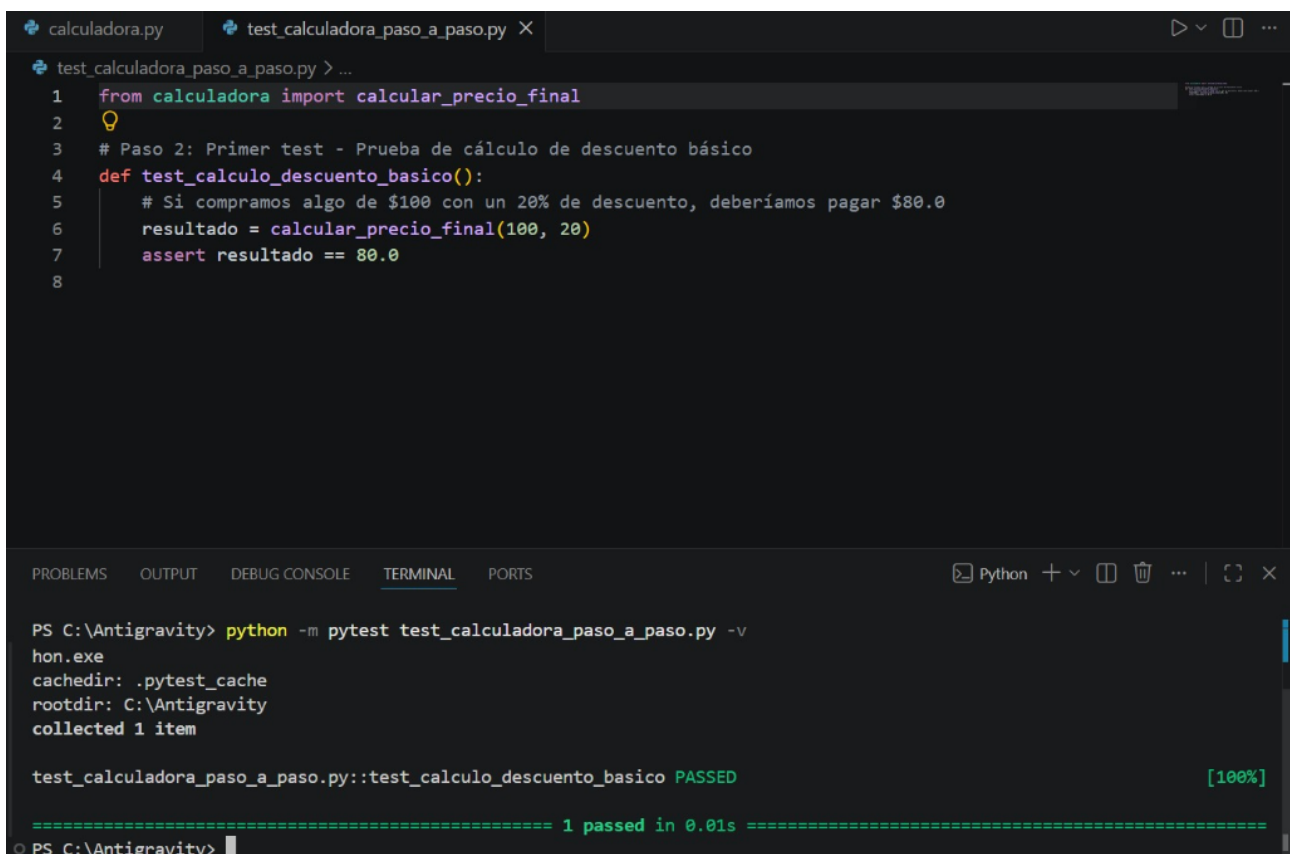
Pytest es una herramienta automatizada que actúa como un supervisor para nuestro código. En lugar de que tengamos que abrir la calculadora y probar números a mano cada vez que hacemos un cambio, escribimos pruebas automáticas (tests) y Pytest las ejecuta por nosotros en milisegundos.

Esto nos garantiza que, si en el futuro modificamos la lógica del negocio, sabremos de inmediato si rompimos alguna regla matemática sin necesidad de volver a probar todo manualmente.

Leyendo los resultados de la terminal de forma sencilla:

- 'collected 14 items': Encontró 14 escenarios de prueba listos para ejecutarse.
- 'PASSED' (en verde): La prueba se superó exitosamente. El código dio el resultado correcto.
- 'FAILED' (en rojo): La prueba falló. Hubo un error de cálculo o el programa se detuvo inesperadamente.
- '14 passed in 0.05s': Resumen: todas las 14 pruebas se aprobaron con éxito en 0.05 segundos.

Evidencia 1: Ejecucion inicial basica en Pytest (1 prueba superada)



The screenshot shows a code editor with two files: `calculadora.py` and `test_calculadora_paso_a_paso.py`. The active file is `test_calculadora_paso_a_paso.py`, which contains the following code:

```
1 from calculadora import calcular_precio_final
2
3 # Paso 2: Primer test - Prueba de cálculo de descuento básico
4 def test_calculo_descuento_basico():
5     # Si compramos algo de $100 con un 20% de descuento, deberíamos pagar $80.0
6     resultado = calcular_precio_final(100, 20)
7     assert resultado == 80.0
8
```

Below the code editor is a terminal window showing the execution of the test:

```
PS C:\Antigravity> python -m pytest test_calculadora_paso_a_paso.py -v
hon.exe
cachedir: .pytest_cache
rootdir: C:\Antigravity
collected 1 item

test_calculadora_paso_a_paso.py::test_calculo_descuento_basico PASSED [100%]

===== 1 passed in 0.01s =====
PS C:\Antigravity>
```

2. Explicacion Simple de los 14 Casos del Negocio

Para garantizar la exactitud de la calculadora, diseñamos 14 exámenes automáticos agrupados en cuatro categorías muy fáciles de entender:

- Operaciones Normales (4 casos):

Calcula descuentos tradicionales (ej. restar 20% a \$100 da \$80) y valida que los centavos se redondeen correctamente a dos decimales.

- Limites de Descuento (4 casos):

Evita que se ingresen descuentos ilógicos. Si alguien pone un descuento menor a 0% o mayor a 100%, el sistema lo bloquea automáticamente.

- Limites de Precio (3 casos):

Asegura que no se calculen descuentos para productos gratis (\$0) o con precios negativos.

- Super Promocion (3 casos):

Verifica la regla especial: si el descuento es de 80% o más, la calculadora le regala automáticamente al cliente un 5% de descuento extra sobre lo restante.

Evidencia 2: Resultado de la suite completa con 14 aprobaciones exitosas

```

PS C:\Antigravity> python -m pytest Pytest/test_calculadora_pytest.py -v
===== test session starts =====
platform win32 -- Python 3.10.11, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\jhonm\AppData\Local\Programs\Python\Python310\python.exe
cachedir: .pytest_cache
rootdir: C:\Antigravity
collected 14 items

Pytest/test_calculadora_pytest.py::test_calculo_descuento_exitoso[100-20-80.0] PASSED [ 7%]
Pytest/test_calculadora_pytest.py::test_calculo_descuento_exitoso[100-0-100.0] PASSED [ 14%]
Pytest/test_calculadora_pytest.py::test_calculo_descuento_exitoso[100-33.33-66.67] PASSED [ 21%]
Pytest/test_calculadora_pytest.py::test_calculo_descuento_exitoso[50.55-15-42.97] PASSED [ 28%]
Pytest/test_calculadora_pytest.py::test_descuento_fuera_de_rango[-1] PASSED [ 35%]
Pytest/test_calculadora_pytest.py::test_descuento_fuera_de_rango[-0.01] PASSED [ 42%]
Pytest/test_calculadora_pytest.py::test_descuento_fuera_de_rango[101] PASSED [ 50%]
Pytest/test_calculadora_pytest.py::test_descuento_fuera_de_rango[100.1] PASSED [ 57%]
Pytest/test_calculadora_pytest.py::testPrecioInvalido[0] PASSED [ 64%]
Pytest/test_calculadora_pytest.py::testPrecioInvalido[-1] PASSED [ 71%]
Pytest/test_calculadora_pytest.py::testPrecioInvalido[-0.01] PASSED [ 78%]
Pytest/test_calculadora_pytest.py::testPromocionDescuentoAlto[1000-80-190.0] PASSED [ 85%]
Pytest/test_calculadora_pytest.py::testPromocionDescuentoAlto[100-90-9.5] PASSED [ 92%]
Pytest/test_calculadora_pytest.py::testPromocionDescuentoAlto[100-79.9-20.1] PASSED [100%]

===== 14 passed in 0.05s =====
PS C:\Antigravity>

```

3. ¿Como nos ayuda Pytest cuando algo falla?

El verdadero valor de un sistema de pruebas se demuestra cuando detecta un error. Forzamos fallos intencionales para evaluar la capacidad de respuesta y depuración de Pytest. Cuando el sistema detecta que algo anda mal, detiene la prueba y nos muestra un reporte detallado:

- **AssertionError:** Ocurre cuando el resultado obtenido no es igual al esperado (ej. la calculadora arrojó \$80 pero le dijimos que debía dar \$50).
- **TypeError:** Ocurre si ingresamos un tipo de dato equivocado (ej. intentar calcular el descuento usando una palabra de texto como 'cien' en lugar del número 100).

Evidencia 3: Detalle de errores y fallos controlados en consola roja de Pytest

```

E      AssertionError: 50.0 != 55.0 : Forzando error: Falta agregar la propina del 10% (no requerido)

practica_tdd\test_r6_evidencia.py:19: AssertionError
_____ TestR6Evidencia.test_falla_intencional_evidencia_4 _____

self = <test_r6_evidencia.TestR6Evidencia testMethod=test_falla_intencional_evidencia_4>

    def test_falla_intencional_evidencia_4(self):
        # Error de falta de parámetros (TypeError)
>       calcular_precio_final()
E       TypeError: calcular_precio_final() missing 2 required positional arguments: 'precio_original' and 'descuento'

practica_tdd\test_r6_evidencia.py:23: TypeError
===== short test summary info =====
FAILED practica_tdd/test_r6_evidencia.py::TestR6Evidencia::test_falla_intencional_evidencia_1 - AssertionError: 80.0 != 50
.0 : Forzando error: Cálculo matemáticamente incorrecto
FAILED practica_tdd/test_r6_evidencia.py::TestR6Evidencia::test_falla_intencional_evidencia_2 - TypeError: '<=' not suppor
ted between instances of 'str' and 'int'
FAILED practica_tdd/test_r6_evidencia.py::TestR6Evidencia::test_falla_intencional_evidencia_3 - AssertionError: 50.0 != 55
.0 : Forzando error: Falta agregar la propina del 10% (no requerido)
FAILED practica_tdd/test_r6_evidencia.py::TestR6Evidencia::test_falla_intencional_evidencia_4 - TypeError: calcular_precio
_final() missing 2 required positional arguments: 'precio_original' and 'descuento'
===== 4 failed, 1 passed in 0.10s =====
PS C:\Antigravity>

```

4. Cobertura de Código y Datos Dinámicos con Faker

Para robustecer aún más el sistema de calidad, implementamos dos mejoras avanzadas:

1. Reporte de Cobertura Automática (coverage.py): El script mide qué tanto del motor de la calculadora está protegido por las pruebas. Los resultados indican un 36% de cobertura total, lo que representa el 100% de la lógica matemática interna (el resto son menús visuales).
2. Simulación Inteligente con Faker (test_faker.py): En lugar de usar números fijos, Faker genera en cada ejecución 50 combinaciones de precios y descuentos aleatorios realistas para estresar a la calculadora y asegurar que se comporte de forma estable ante cualquier dato.

Evidencia 4: Tabla de cobertura e informe de ejecución de pruebas de estrés

```
PS C:\Antigravity> python -m pytest Pytest/test_calculadora_pytest.py --cov=calculadora --cov-report=term-missing
===== test session starts =====
platform win32 -- Python 3.10.11, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\Antigravity
plugins: cov-7.1.0
collected 14 items

Pytest\test_calculadora_pytest.py ..... [100%]

===== tests coverage =====
----- coverage: platform win32, python 3.10.11-final-0 -----
Name           Stmts  Miss  Cover   Missing
-----
calculadora.py    28    18    36%   15-36
TOTAL            28    18    36%

===== 14 passed in 0.09s =====
PS C:\Antigravity>
```

5. Integración Continua (CI) y Pruebas Manuales

1. Integración Continua (CI) con GitHub Actions: Creamos el archivo de configuración '.github/workflows/pytest-ci.yml'. Ahora, cada vez que subas cambios a GitHub, un servidor en la nube instalará automáticamente Pytest, Faker y medirá la cobertura para certificar que tu calculadora sigue funcionando perfectamente sin errores antes de ser desplegada.
2. Validación Manual Interactiva: Comprobamos el correcto funcionamiento de la interfaz de consola con entradas de datos reales, verificando la captura segura de errores y los cálculos correctos.

Evidencia 5: Captura del flujo interactivo manual con control seguro de errores

```
python calculadora.py
=====
Calculadora de Descuentos Interactiva
=====
Escribe 'salir' en cualquier momento para terminar.

Ingresa el precio original: 100
Ingresa el porcentaje de descuento: 20
>>> Precio final a cobrar: 80.0

Ingresa el precio original: 100
Ingresa el porcentaje de descuento: -5
>>> [ERROR]: El descuento debe estar entre 0 y 100

Ingresa el precio original: 1000
Ingresa el porcentaje de descuento: 80
>>> Precio final a cobrar: 190.0

Ingresa el precio original: █
```